ABAP Moderno: Tipos de Dados e Conversões

Estratégias para um Código Seguro, Performático e Livre de Dumps



Objetivos de Aprendizagem



Identificar e selecionar Tipos Primitivos (I, P, C, STRING) considerando memória e precisão.



Diferenciar Conversão Implícita de Explícita e reconhecer seus riscos.



Utilizar o operador CONV para conversões seguras e legíveis.



Prevenir erros de Overflow e Truncation em ambientes produtivos.

NotebookLM

Escolha Estratégica de Tipos Primitivos

Numéricos

- I (Integer): 4 bytes.
  Ideal: Contadores e Índices.
  Risco: Arredonda decimais.
- P (Packed): Ponto fixo.
  Ideal: Dinheiro e Pesos.
  Garante precisão exata.
- F (Float): Ponto flutuante.
  Ideal: Cálculos científicos.
  Risco: Nunca usar para saldo bancário.

Texto

- C (Char): Tamanho fixo.
  Preenche espaço vazio na memória.
  Uso: Códigos/IDs.
- STRING: Tamanho dinâmico. Aloca só o necessário.
  Uso: Descrições e Logs (Padrão Moderno).

Data e Hora

- D (Date): Formato interno AAAAMMDD. Permite aritmética de dias.
- T (Time): Formato interno HHMMSS. Permite aritmética de segundos.

NotebookLM

A Ilusão da ‘Mágica’ do Compilador

SAP Fiori Quartz Light

O ABAP é permissivo: tenta converter tipos automaticamente na atribuição (=).

Isso gera ‘Bug Silencioso’: o código compila, mas falha ou corrompe dados em execução.

```
DATA: lv_num TYPE i.

lv_num = '100'.      " OK: Vira 100
lv_num = '100.9'.    " PERIGO: Arredonda para 101
lv_num = '100,9'.    " ERRO CRÍTICO: Dump (Depende do usuário)
```

⚠ A conversão implícita esconde a intenção do código.

NotebookLM

Principais Causas de Dumps em Produção

Truncation (Corte Silencioso)

- **Conceito**: Mover texto longo para variável curta.
- **Exemplo**: 'Bananas' (7 chars) → CHAR(3)
- **Resultado**: 'Ban' (Dados perdidos sem aviso).

Overflow (Estouro Numérico)

- **Conceito**: Número maior que a capacidade da variável.
- **Exemplo**: Jogar valor milionário em variável pequena.
- **Resultado**: DUMP (CX_SY_CONVERSION_OVERFLOW)). O programa cai.

NotebookLM

Conversão Explícita: Código Limpo e Robusto

O operador construtor **CONV** deixa clara a transformação e permite controle total.

```
DATA(var) = CONV tipo_destino( valor_origem ).
```

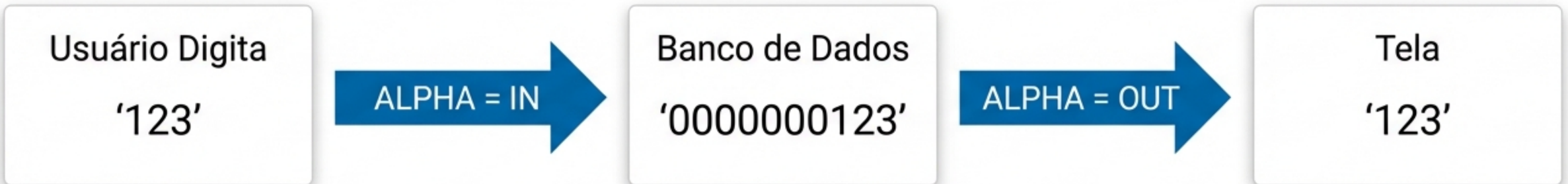
- **Legibilidade**: Óbvio para quem lê que o tipo mudou.
- **Inline**: Uso direto em métodos e concatenações.
- **Segurança**: Facilita o tratamento de exceções (TRY...CATCH).

```
lo_calc->somar( iv_val1 = CONV i( '10' ) ).
```

NotebookLM

Tratamento de Identificadores (Materiais e Clientes)

No SAP, IDs (ex: Cliente) são textos que exigem zeros à esquerda no Banco de Dados. Sem Alpha, o SELECT falha.



IN:

```
|{ '123' ALPHA = IN }|
```

OUT:

```
|{ '0000000123' ALPHA = OUT }|
|{ '0000000123' ALPHA = OUT }|
```

NotebookLM

Exemplo Real: Processando Arquivos Externos

Cenário: Recebimento de valor em String ('150.55') vindo de API/Arquivo.

```
DATA(lv_string) = '150.55'.

TRY.
    " Converte String para KBETR (Moeda) explicitamente
    DATA(lv_valor) = CONV kbetr( lv_string ).

CATCH cx_sy_conversion_no_number.
    " Tratamento de erro robusto
    out->write( 'Erro: O texto não é numérico.' ).
ENDTRY.
```

Uso de TRY...CATCH evita o Dump fatal.

NotebookLM

Tabela Comparativa: Melhores Práticas

Cenário	Tipo Recomendado	Tipo a Evitar	Por quê?
Dinheiro	Packed (P)	Float (F)	Float possui imprecisão binária (perda de centavos).
Contadores	Integer (I)	Packed (P)	Inteiros são processados mais rápido pela CPU.
Texto	String	Char (C)	String economiza memória e evita truncamento.
IDs (Mat/Cli)	Char / Numc	Integer	IDs possuem zeros à esquerda que Inteiro remove.

NotebookLM

Glossário Técnico

- **Type Casting**: Processo de transformar um tipo de dado em outro.
- **Implicit Conversion**: Conversão automática e arriscada feita pelo compilador.
- **Explicit Conversion (CONV)**: Conversão forçada via código, segura e legível.
- **Overflow**: Erro crítico (Dump) quando o número excede a capacidade da variável.
- **Truncation**: Perda de dados (corte) ao mover texto longo para variável curta.
- **Alpha Conversion**: Rotina padrão SAP para gerenciar zeros à esquerda em IDs.

NotebookLM

Teste seu Conhecimento

Q1: O que acontece se converter a string ‘ABC’ para Inteiro usando CONV sem tratamento de erro?

Q2: Qual a principal diferença de alocação de memória entre CHAR e STRING?

Q3: Qual a sintaxe de String Template para adicionar zeros à esquerda em um número de material (Alpha Input)?

NotebookLM

A clareza do código define a estabilidade da solução.

Revise seus programas Z. Substitua conversões implícitas por CONV e garanta a integridade dos seus dados hoje.

NotebookLM